

VISVESWARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belgaum 590014, Karnataka



MINI PROJECT REPORT ON

“CONGESTION CONTROL”

Submitted in partial fulfillment for the requirement of 6th semester

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

By

ABHILASH MADHAV M (1AH19CS002)

MOHIT KUMAR M (1AH18CS053)

NANDAN R (1AH18CS057)

Under the Guidance of

Ms. GANGA BM

Assistant Professor

Department of CSE



Department of Computer Science and Engineering
ACS COLLEGE OF ENGINEERING

#74, Kambipura, Mysore Road, Bengaluru – 560074

2021– 2022

ACS COLLEGE OF ENGINEERING

#74, Kambipura, Mysore Road, Bengaluru – 560074

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

Certified that the Mini Project on topic **“CONGESTION CONTROL”** has been successfully submitted by **ABHILASH MADHAV M(1AH19CS002), MOHIT KUMAR M(1AH18CS053)** and **NANDAN R(1AH18CS057)** a bonafide student of **ACS COLLEGE OF ENGINEERING** affiliated to **Visveswaraya Technological University, Belgaum** during the year 2021-2022. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report submitted to the department. The Mini Project report has been approved as it satisfies the academic requirements in respect of mini project work as prescribed in 6th semester.

Signature of the Guide

Ms. Ganga BM
Assistant Professor, CSE

Signature of the HOD

Dr. Mareeswari .V
Prof & HOD, CSE

Name of the examiners

1. _____

2. _____

Signature with date

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and respect to the ACS College of Engineering, Bengaluru for providing me an opportunity to carry out my mini project report.

I express my deep regards to our honourable chairman **Dr. A. C. Shanmugam** for providing me an opportunity to fulfil my ambition in this prestige institute.

I would like to express my immense gratitude to **Dr. M. S. Murali**, Principal, ACS College of Engineering, Bengaluru, for his timely help and inspiration during the tenure of the course.

I express our sincere regards and thanks to **Dr.Mareeswari V**, Professor & HOD, Computer Science and Engineering, ACSCE, Bengaluru for the encouragement and support throughout the work.

With profound sense of gratitude, I acknowledge the guidance, support and encouragement by my guide **Ms Ganga BM**, Assistant Professor of Computer Science and Engineering, ACSCE, Bengaluru.

ABHILASH MADHAV M

(1AH19CS002)

MOHIT KUMAR M

(1AH18CS053)

NANDAN R

(1AH18CS057)

LIST OF CONTENTS

Sl. No

Chapter page no.

Acknowledgement	i
List of Figures.....	ii
Abstract	iii
1. Introduction	1
1.1 Introduction about CG.....	1
1.2 OpenGL fundamentals	1
1.3 Basic OpenGL operation.....	2
2. Project Description	3
2.1 What is congestion control?	3
2.2 Leaky Bucket Algorithm	3
2.3 Token Bucket Algorithm.....	4
3. System Software Requirements.....	5
3.1 Hardware Requirements.....	5
3.2. Software Requirements.....	5
4. Implementation.....	6
4.1 Operating System and Languages Used.	6
4.2 Data Structures	6
4.3 Functions Used	6
4.4 Source Code	7
5. Results	29
5.1 Sample output.....	29
6. Conclusions	31
7. Bibliography	32
7.1 Text Books	32
7.2 WebLinks	32

LIST OF FIGURES

Slno	Name	Page No
1	OpenGL block diagram Fig 1.1	2
2	Leaky Bucket Algorithm Fig2.2.1	3
3	Sample Output	29
	Fig5.1.1 BackPressure	29
	Fig5.1.2ChokePacket	29
	Fig 5.1.3Implicit Signaling	30
	Fig 5.1.4 Explicit Signaling	31

ABSTRACT

Main aim of this Mini Project is to illustrate the concepts and usage of Congestion Control in OpenGL. In data networking and queuing theory, network congestion occurs when a link or node is carrying so much data that its quality of service deteriorates. When more packets were sent than could be handled by intermediate routers, the intermediate routers discarded many packets, expecting the end points of the network to retransmit the information. When this packet loss occurred, the end points sent *extra* packets that repeated the information lost, doubling the data rate sent, exactly the opposite of what should be done during congestion. This pushed the entire network into a 'congestion collapse' where most packets were lost and the resultant throughput was negligible. We have used input devices like mouse and keyboard to interact with program.

CHAPTER 1

INTRODUCTION

Introduction about CG

Computer Graphics is concerned with all aspects of producing pictures or images using a computer. Nowadays we can create images by computer that are indistinguishable from photographs of real objects.

As a software interface for graphics hardware, OpenGL's main purpose is to render two- and three-dimensional objects into a frame buffer.

These objects are described as sequences of vertices or pixels.

OpenGL performs several processing steps on this data to convert it to pixels to form the final desired image in the frame buffer.

OpenGL Fundamentals

This section explains some of the concepts inherent in OpenGL.

Primitives and Commands

OpenGL draws primitives—points, line segments, or polygons—subject to several selectable modes.

You can control modes independently of each other; that is, setting one mode doesn't affect whether other modes are set. Primitives are specified, modes are set, and other OpenGL operations are described by issuing commands in the form of function calls.

Primitives are defined by a group of one or more vertices. A vertex defines a point, an endpoint of a line, or a corner of a polygon where two edges meet. Data is associated with a vertex, and each vertex and its associated data are processed independently, in order, and in the same way. The type of clipping depends on which primitive the group of vertices represents.

Commands are always processed in the order in which they are received, although there may be an indeterminate delay before a command takes effect. This means that each primitive is drawn completely before any subsequent command takes effect. It also means that state-querying commands return data that's consistent with complete execution of all previously issued OpenGL commands.

Basic OpenGL Operation

The figure shown below gives an abstract, high-level block diagram of how

OpenGL processes data. In the diagram, commands enter from the left and proceed through what can be thought of as a processing pipeline. Some commands specify geometric objects to be drawn, and others control how the objects are handled during the various processing stages.

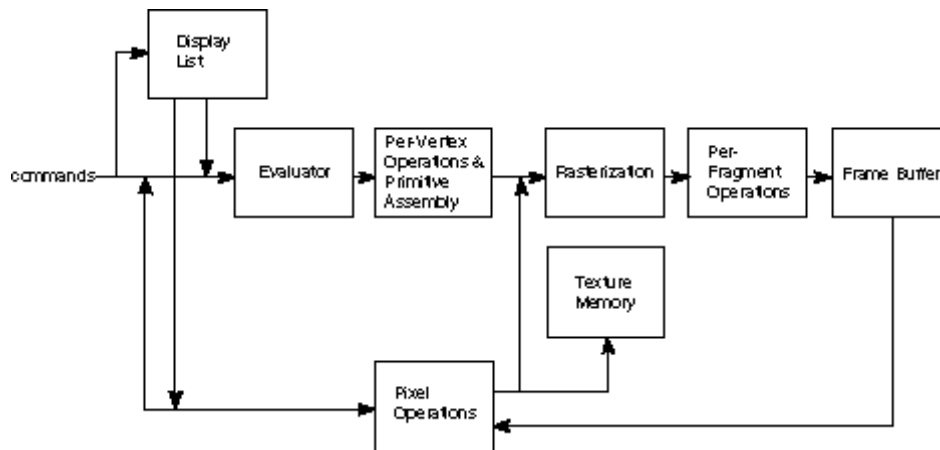


Figure-1.1 OpenGL Block Diagram

As shown by the first block in the diagram, rather than having all commands proceed immediately through the pipeline, you can choose to accumulate some of them in a display list for processing at a later time.

Rasterization produces a series of frame buffer addresses and associated values using a two-dimensional description of a point, line segment, or polygon.

Each fragment so produced is fed into the last stage,

per-fragment operations, which performs the final operations on the data before it's stored as pixels in the frame buffer. These operations include conditional updates to the frame buffer based on incoming and previously stored z-values (for z-buffering) and blending of incoming pixel colors with stored colors, as well as masking and other logical operations on pixel values.

All elements of OpenGL state, including the contents of the texture memory and even of the frame buffer, can be obtained by an OpenGL application.

CHAPTER 2

PROJECT DESCRIPTION

What is congestion?

A state occurring in network layer when the message traffic is so heavy that it slows down network response time.

Effects of Congestion

- As delay increases, performance decreases.
- If delay increases, retransmission occurs, making situation worse.

There is two congestion control algorithm which is as follows:

Leaky Bucket

- The leaky bucket algorithm discovers its use in the context of network traffic shaping or rate-limiting. The algorithm allows controlling the rate at which a record is injected into a network and managing burstiness in the data rate.
- A leaky bucket execution and a token bucket execution are predominantly used for traffic shaping algorithms. This algorithm is used to control the rate at which traffic is sent to the network and shape the burst traffic to a steady traffic.

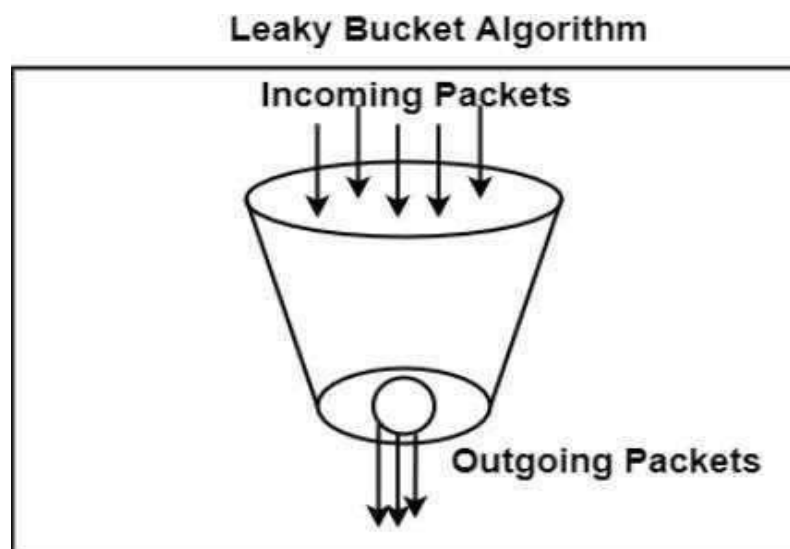


Figure-2.2.1

Token Bucket Algorithm

The token bucket algorithm adds a token to the bucket each $1/r$ seconds. The volume of the bucket is b tokens. When a token appears, and the bucket is complete, the token is discarded. If a packet of n bytes appears and n tokens are deleted from the bucket, the packet is forwarded to the network.

When a packet of n bytes appears but fewer than n tokens are available. No tokens are removed from the bucket in such a case, and the packet is considered non-conformant. The non-conformant packets can either be dropped or queued for subsequent transmission when sufficient tokens have accumulated in the bucket.

CHAPTER 3

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENT:

- GRAPHICS SYSTEM: A graphics system produces the visuals that allow you to interact with your computer.
- RAM: Pentium P4 with 256 of Ram (Min).
- MOUSE.
- KEYBOARD 108 STANDARD.
- MONITOR RESOLUTION 800X600.

SOFTWARE REQUIREMENT:

- MICROSOFT VISUAL C++: A Visual C++ compiler is required for compiling the source code to make the executable file which can then be directly executed
- OPENGL: OpenGL is a library for doing computer graphics. By using it, you can create interactive applications which render high-quality color images composed of 3D geometric objects and images.

CHAPTER 4

IMPLEMENTATION

Operating System And Language Used

The Operating System Used in the project is Windows 10 and the Language used for the project is C++.

Data Structures

The basic idea is that, to avoid congestion, extra information can be broadcasted to the group periodically to tell the routers along the tree to maintain certain data structures in their memories

Functions Used

- glutInit() : interaction between the windowing system and OPENGGL is initiated
- glutInitDisplayMode() : used when double buffering is required and depth information is required
- glutCreateWindow() : this opens the OPENGGL window and displays the title at top of the window
- glutInitWindowSize() : specifies the size of the window
- glutInitWindowPosition() : specifies the position of the window in screen co-ordinates
- glutKeyboardFunc() : handles normal ascii symbols
- glutSpecialFunc() : handles special keyboard keys
- glutReshapeFunc() : sets up the callback function for reshaping the window
- glutIdleFunc() : this handles the processing of the background
- glutDisplayFunc() : this handles redrawing of the window

- glutMainLoop() : this starts the main loop, it never returns
- glViewport() : used to set up the viewport
- glVertex3fv() : used to set up the points or vertices in three dimensions
- glColor3fv() : used to render color to faces
- glFlush() : used to flush the pipeline
- glutPostRedisplay() : used to trigger an automatic redrawal of the object
- glMatrixMode() : used to set up the required mode of the matrix
- glLoadIdentity() : used to load or initialize to the identity matrix
- glTranslatef() : used to translate or move the rotation centre from one point to another in three dimensions
- glRotatef() : used to rotate an object through a specified rotation angle

Source Code

```
#include <windows.h>
#include<string.h>
#include<stdarg.h>
#include<stdio.h>
#include <glut.h>
void *font;
void *currentfont;
static double x=0.0,x1=-3.5,x2=3.5,y1=-1.4,y2=-1.4,x3=-3.5,y3=1.3;
static double move=-60;
static bool
goDown=false,goup=false,down=false,congested=false,remote=false;
void setFont(void *font)
{
    currentfont=font;
}
```

```
void drawstring(float x,float y,float z,char *string)
{
    char *c;
    glRasterPos3f(x,y,z);

    for(c=string;*c!='\0';c++)
    {
        glColor3f(0.0,1.0,1.0);
        glutBitmapCharacter(currentfont,*c);
    }
}

void stroke_output(GLfloat x, GLfloat y, char *format,...)
{
    va_list args;
    char buffer[200], *p;
    va_start(args, format);
    vsprintf(buffer, format, args);
    va_end(args);
    glPushMatrix();
    glTranslatef(x, y,2);
    glScaled(0.003, 0.005, 0.005);
    for (p = buffer; *p; p++)
        glutStrokeCharacter(GLUT_STROKE_ROMAN, *p);
    glPopMatrix();
}

void cloud(){
    //vertical eclipse
    glPushMatrix();
    glTranslatef(0.0,0.0,0.0);
    glScaled(1,3.5,0.1);
    glColor3f(1,1,1);
    glutSolidSphere(1,100,100);
    glPopMatrix();
    //horizotal eclipse
    glPushMatrix();
    glTranslatef(0.0,0.0,0.0);
    glScaled(3,2.65,0.1);
    glColor3f(1,1,1);
    glutSolidSphere(1,100,100);
}
```

```
        glPopMatrix();
        glPushMatrix();
        glScaled(3.5,1.8,0.1);
        glRotatef(60,0.0f,1.0f,1.0f);
        glTranslatef(0.0,0.0,0.0);
        glColor3f(1,1,1);
        glutSolidSphere(1,100,100);
        glPopMatrix();
        glPushMatrix();
        glScaled(1.5,3.5,0.1);
        glRotatef(50,0.0f,1.0f,1.0f);
        glTranslatef(0.0,0.0,0.0);
        glColor3f(1,1,1);
        glutSolidSphere(1,100,100);
        glPopMatrix();
    }
void router(float x6, float y6,float z6)
{
    glColor3f(0.2,0.3,1);
    glPushMatrix();
    glScaled(0.2,1.0,0.1);
    glRotatef(91,1.0f,0.0f,0.0f);
    glTranslatef(x6,y6,z6);
    glutSolidTorus(0.2,1.5,100,100);
    glPopMatrix();
}
void dte(float x1, float x2)
{
    glPushMatrix();
    glTranslatef(x1,x2,0.1);
    glScaled(1,0.1,0.1);
    glColor3f(1,1,1);
    glRotatef(0,0.0f,1.0f,0.0f);
    glutWireCube(1.5);
    glPopMatrix();
}
void line()
```

```
{
glPushMatrix();
    glBegin(GL_LINE_LOOP);
        glColor3f(1,0,0);
        glVertex3f(0.1,-0.2,1);
        glVertex3f(0.0,0,1);
        glVertex3f(2,1.3,1);
        glVertex3f(2.1,1.1,1);
    glEnd();
glPopMatrix();
}

void line1()
{
glPushMatrix();
    glBegin(GL_LINE_LOOP);
        glColor3f(1,0,0);
        glVertex3f(0.2,2.5,1);
        glVertex3f(0.1,2.3,1);
        glVertex3f(2.1,1.1,1);
        glVertex3f(2.3,1.2,1);
    glEnd();
glPopMatrix();
}

void rack(float x1, float x2)
{
glPushMatrix();
    glScaled(0.35,.05,1.0);
    glTranslatef(x1,x2,0.1);
    glColor3f(0,0,0.3);
    glutSolidCube(1.1);
    glPopMatrix();
}

void rack1(float x1, float x2)
{
glPushMatrix();
    glScaled(1.75,.1,1.0);
    glTranslatef(x1,x2,0.3);
```

```
        glColor3f(0,0,1.3);
        glutSolidCube(0.5);
        glPopMatrix();
    }
    void window(float x1, float x2)
    {
        glPushMatrix();
            glScaled(0.5,.3,1.0);
            glTranslatef(x1,x2,0.4);
            glColor3f(0.1,0.1,0.1);
            glutSolidCube(0.5);
            glPopMatrix();
        }
    void top()
    {
        glBegin(GL_TRIANGLES);
        /* Front */
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f( 0.0f, 1.0f, 0.0f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(-0.5f,-0.5f, 0.5f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f( 0.5f,-0.5f, 0.5f);
        /* Left */
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(0.0f,1.0f,0.0f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(-0.5f,-0.5f,0.5f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(-0.5f,-0.5,-0.5f);
        /* Right */
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(0.0f,1.0f,0.0f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(0.5f,-0.5f,0.5f);
        glColor3f(1.0f,1.0f,1.0f);
        glVertex3f(0.5f,-0.5,-0.5f);
```

```
/* Back */
glColor3f(1.0f,1.0f,1.0f);
glVertex3f( 0.0f, 1.0f, 0.0f);
glColor3f(1.0f,1.0f,1.0f);
glVertex3f(-0.5f,-0.5f, -0.5f);
glColor3f(1.0f,1.0f,1.0f);
glVertex3f( 0.5f,-0.5f, -0.5f);
glEnd();
}
void animate();
void network()
{
    glClearColor(0.0,0,0,0.0);
    //glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f,0.0f,-13.0f);
    // glRotatef(ang,1.0f,1.0f,0.0f);
    //Server
    glPushMatrix();
    glScaled(0.8,2.0,0.8);
    glTranslatef(5,0.8,0.0);
    glColor3f(0,1.5,1.5);
    glutSolidCube(1);
    glPushMatrix();
    glScaled(0.5,.1,1.0);
    glTranslatef(.0,3.5,0.01);
    glColor3f(0.3,0.3,0.3);
    glutSolidCube(1.5);
    glPopMatrix();
    rack1(.0,1);
    rack1(.0,2);
    rack1(.0,0);
    rack(-1,-4);
    rack(1,-4);
    rack(-1,-6);
    rack(1,-6);
    rack(-1,-8);
```

```
rack(1,-8);
glPopMatrix();
//Home
glPushMatrix();
glScaled(0.8,1.0,0.8);
glTranslatef(4.8,-2.5,0.0);
glColor3f(1,1,0);
glutSolidCube(1);
glPushMatrix();
glScaled(.7,.1,1.0);
glTranslatef(0.0,4.5,0.2);
glColor3f(1.3,1.3,1.3);
glutSolidCube(1.5);
glPopMatrix();
glPushMatrix();
glScaled(0.12,.3,.75);
glTranslatef(0,-.75,0.02);
glColor3f(0.3,0.3,0.3);
glutSolidCube(1.5);
glPopMatrix();
// Top
glPushMatrix();
glTranslatef(0,.9,0);
top();
glPopMatrix();
window(-.6,-.3);
glPopMatrix();
//Office Building
glPushMatrix();
glScaled(0.8,2.0,0.8);
glTranslatef(-4.8,-1,0.0);
glColor3f(2,.5,.5);
glutSolidCube(1);
window(-.5,1.3);
window(.5,1.3);
window(-.5,0.5);
window(.5,0.5);
```

```
window(-.5,-0.3);
window(.5,-0.3);
window(-.5,-1.1);
window(.5,-1.1);
glPopMatrix();
// Remote User
glPushMatrix();
glTranslatef(-4.0,1.5,0.0);
glScaled(0.3,.3,.3);
glColor3f(1.3,1.3,1.3);
glutSolidCube(0.5);
glPopMatrix();
// Routers inside cloud
//router 5
router(10,10,1.15);
//router 2
router(-10,10,-1.3);
//router 1
router(-10,10,1.2);
//router 3
router(0,10,-2.5);
    router(0,10,0);
    //router 5
router(0,10,2.5);
    //router 4
router(10.6,10,-1.2);
// Link 1
glPushMatrix();
glTranslatef(-2.1,0.0,0.5);
glScaled(0.1,1.8,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();

// Link 2
glPushMatrix();
glTranslatef(-2,1.15,0);
```

```
line();
glPopMatrix();

// Link 3 to server
glPushMatrix();
glTranslatef(1.2,1.7,0.5);
glRotatef(57,0,0,1);
glScaled(0.15,1.7,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();
// Link 4
glPushMatrix();
glTranslatef(0.85,0.4,0.5);
glRotatef(120,0,0,1);
glScaled(0.2,1.7,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();
// Link 5
glPushMatrix();
glTranslatef(0,-2.6,0);
line();
glPopMatrix();
// Link 6
glPushMatrix();
glTranslatef(-2.2,-3.85,0);
line1();
glPopMatrix();
// Link 7(1step data transfer from ...)
glPushMatrix();
glTranslatef(-1.2,-0.85,0.5);
glRotatef(125,0,0,1);
glScaled(0.15,1.6,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();
```

```
// Link 9
glPushMatrix();
glTranslatef(0.0,1.0,0.5);
glScaled(0.1,1.65,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();

//R0 - R6
glPushMatrix();
glTranslatef(0.0,-1.6,0.5);
glScaled(0.1,1.65,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();
//link 10 from home
glPushMatrix();
glTranslatef(1.05,-0.8,0.5);
glRotatef(52,0,0,1);
glScaled(0.15,1.6,0.1);
glColor3f(1,0,0);
glutWireCube(1.5);
glPopMatrix();
// Office Link to Cloud
dte(-3.0,-1.4);
// Home Link to Cloud
dte(3.0,-1.4);
// Server Link to Cloud
dte(3.0,1.1);
//Cloud Call Function
cloud();
glPushMatrix();
//glScaled(1.1,2.0,0.1);
glTranslatef(0.0,0.0,2.0);
setFont(GLUT_BITMAP_TIMES_ROMAN_24);
glColor3f(0,0,0);
drawstring(-0.3,2.5,0.0,"CLOUD");
```

```
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(-2.1,-1.4,0.0,"R1");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(-2.05,0.95,0.0,"R2");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(0,2,0.0,"R3");

    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(-0.1,-0.3,0.0,"R0");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(1.8,0.8,0.0,"R4");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(1.8,-1.35,0.0,"R5");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(0,-2.6,0.0,"R6");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(1,1,1);
    drawstring(3,-2.1,0.0,"HACKER");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(1.5,1.5,1.5);
    drawstring(-3.7,-0.8,0.0,"OFFICE");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(1.5,1.5,1.5);
    drawstring(3,2.3,0.0,"SERVER");
    glPopMatrix();
    //Transtitions all together
    animate();
    glFlush();
    glutSwapBuffers();
}
```

```
void animate(){
    // Company to server Using path 1
    glPushMatrix();
    glTranslatef(x1,y1,0.2);
    glScaled(0.1,0.1,0.1);
    glColor3f(0,0,1);
    glutSolidCube(0.5);
    glPopMatrix();
    if(remote){
        glPushMatrix();
        glTranslatef(x3,y3,0.2);
        glScaled(0.1,0.1,0.1);
        glColor3f(1,0,0.5);
        glutSolidCube(0.5);
        glPopMatrix();
    }
    if(congested)
    {
        glPushMatrix();
        glTranslatef(x2,y2,0.2);
        glScaled(0.1,0.1,0.1);
        glColor3f(4,0,0);
        glutSolidCube(0.5);
        glPopMatrix();
    }
}

void about()
{
    glClearColor(0.5,0.5,0.5,0.0);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f,0.0f,-13.0f);
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    glColor3f(0,0,0);
    drawstring(-4.2,2,0.0,"About Congestion Control");
    setFont(GLUT_BITMAP_TIMES_ROMAN_24);
    drawstring(-3.5,1.5,0.0,"* Congestion is a situation in Communication
```

Networks in which too many packets");

drawstring(-3.5,1,0.0," are present in a part of the subnet, performance degrades.");

drawstring(-3.5,.5,0.0,"* In data networking network congestion occurs when a link or node ");

drawstring(-3.5,0.0,0.0," is carrying so much data that its quality of service deteriorates.");

drawstring(-3.5,-0.5,0.0,"* Congestion control concerns controlling traffic entry into a ");

drawstring(-3.5,-1,0.0," telecommunications network, so as to avoid congestive collapse by attempting ");

drawstring(-3.5,-1.5,0.0," to avoid oversubscription of any of the processing or link capabilities ");

drawstring(-3.5,-2,0.0," of the intermediate nodes and networks and taking resource reducing steps, ");

drawstring(-3.5,-2.5,0.0," such as reducing the rate of sending packets.");

drawstring(-3.5,-3,0.0,"* It should not be confused with flow control, which prevents the sender from overwhelming the receiver.");

glFlush();

glutSwapBuffers();

}

void s()

{

glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

network();

}

void com_ser(){

// No congestion, take the best path

if(!congested && !remote)

{

if(x1>=-3.5 && y1==-1.4 && x1<=-2)

{

x1+=0.1;

}

```
    if(x1>=-2 && y1>=-1.4 && x1<=2 && y1<=1.8)
    {
        x1+=0.1;
        y1+=0.065;
    }

    if(x1>=2 && y1>=1 && x1<=3.8)
    {
        x1+=0.1;
        y1=1.1;
    }

}
//if Congested only from Hacker A

if(congested && x2>=2 && y2>=1 && !remote)
{
    if(x1>=-3.5 && y1== -1.4 && x1<=-2){
        x1+=0.1;
    }
    if(x1>=-2 && y1>=-1.4 && x1<=0 && y1<=0){
        x1+=0.1;
        y1+=0.07;
    }
    if(x1>=-0.05 && x1<=0.05 && y1<=0.05 && y1>=-0.05){
        x1=0;
        y1=0;
    }
    if(x1==0 && y1>=0 && y1<=2.6){

        y1+=0.06;
    }
    if(x1==0 && y1>=2.5)
        goDown=true;
    if(goDown && x1>=0 && x1<=2 && y1<=2.6 && y1>=1 ){
        x1+=0.1;
```

```
    y1-=0.06;
    }
    if(x1>=2 && y1>=1 && x1<=3.8){
        x1+=0.1;
        y1=1.1;
    }
}

// if congested only from Hacker B
if(remote && y3<=1 && !congested)
{
    if(x1>=-3.5 && y1== -1.4 && x1<=-2){
        x1+=0.1;
    }
    if(x1>=-2 && x1<=-0.2 && y1<=-1.4 && y1>=-3){
        x1+=0.1;
        y1-=0.08;
    }
    if(x1>=-0.3 && x1<=-0.2 && y1<=-2.7 && y1>=-3 && goup==false){
        x1=0;
        y1=-3;
        goup=true;
    }
    if(goup && y1<=0){
        x1=0.0;
        y1+=0.1;
    }
    if(goup && x1>=0 && y1>=0 && x1<=2 && y1<=1.8)
    {
        x1+=0.1;
        y1+=0.055;
    }
    if(x1>=2 && y1>=1 && x1<=3.8)
    {
        x1+=0.1;
        y1=1.1;
    }
}
```

```
}  
// if congested from both Hackers  
if(remote && y3<=1 && congested)  
{  
    if(x1>=-3.5 && y1== -1.4 && x1<=-2){  
        x1+=0.1;  
  
    }  
  
    if(x1>=-2 && x1<=-0.2 && y1<=-1.4 && y1>=-3){  
        x1+=0.1;  
        y1-=0.08;  
    }  
    if(x1>=-0.3 && x1<=-0.2 && y1<=-2.7 && y1>=-3 && goup==false){  
        x1=0;  
        y1=-3;  
        goup=true;  
    }  
    if(goup && y1<=2.6 && goDown==false){  
        x1=0.0;  
        y1+=0.1;  
    }  
    if(x1==0 && y1>=2.5)  
        goDown=true;  
    if(goDown && x1>=0 && x1<=2 && y1<=2.6 && y1>=1 ){  
        x1+=0.1;  
        y1-=0.06;  
    }  
    if(x1>=2 && y1>=1 && x1<=3.8){  
        x1+=0.1;  
        y1=1.1;  
    }  
}
```

CHAPTER 5

RESULTS

OUTPUTS

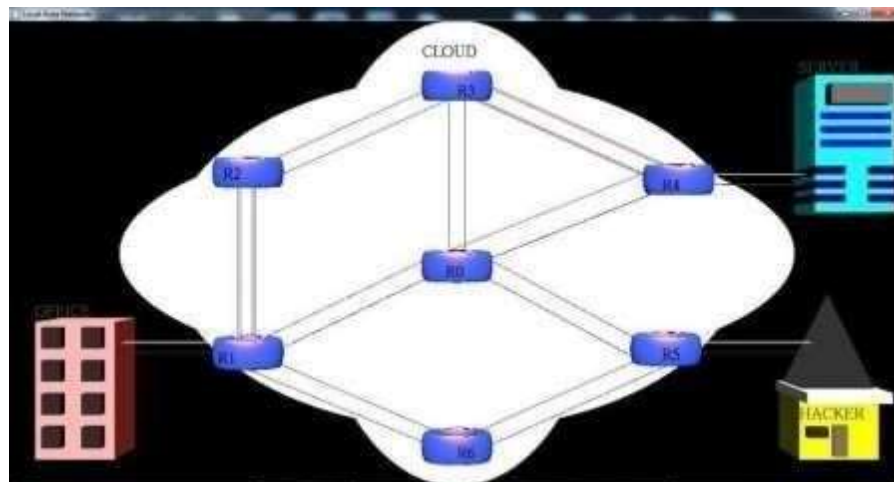


Figure-5.1.1 Backpressure: A special bit in the packet header is set to the router to warn the source when congestion is detected.

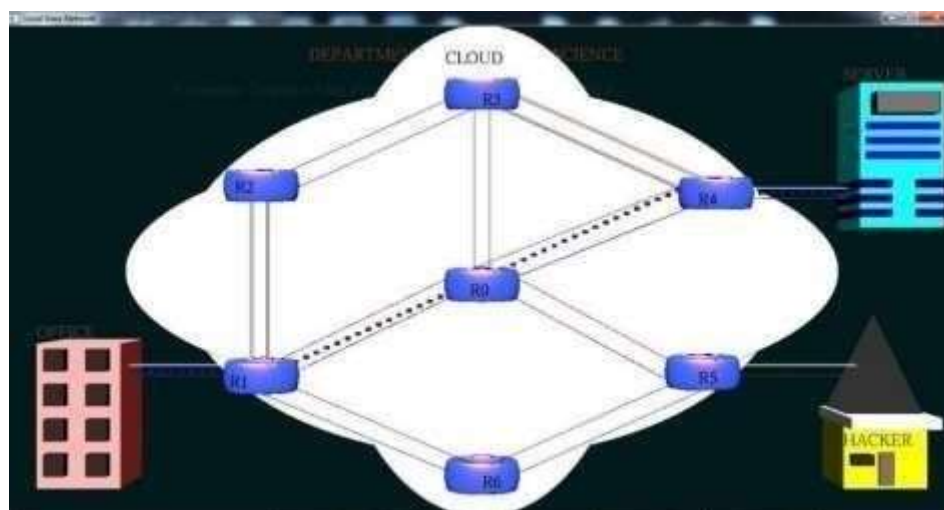


Figure-5.1.2 Choke Packet: A packet is sent by a node to the source to inform it of congestion.

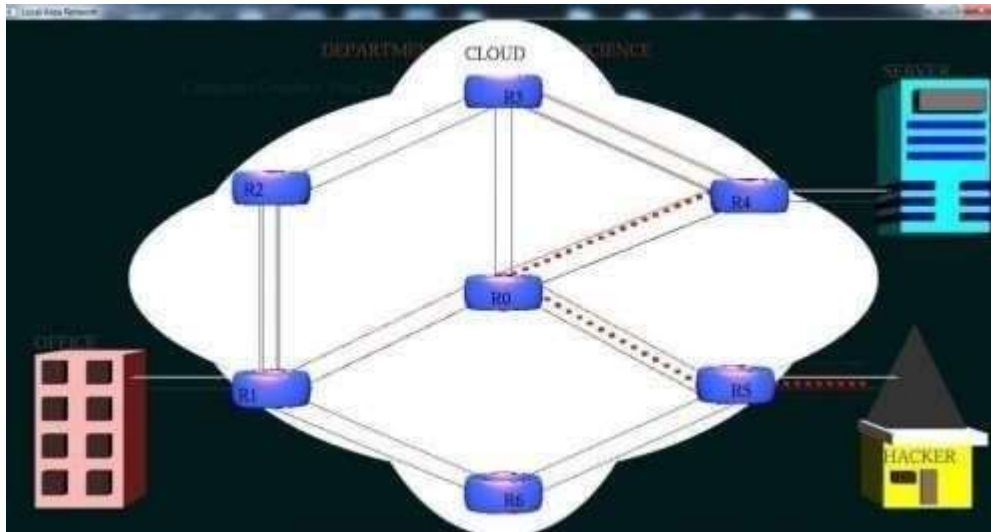


Figure-5.1.3 Implicit Signaling: There is no communication between the congested nodes and the source.

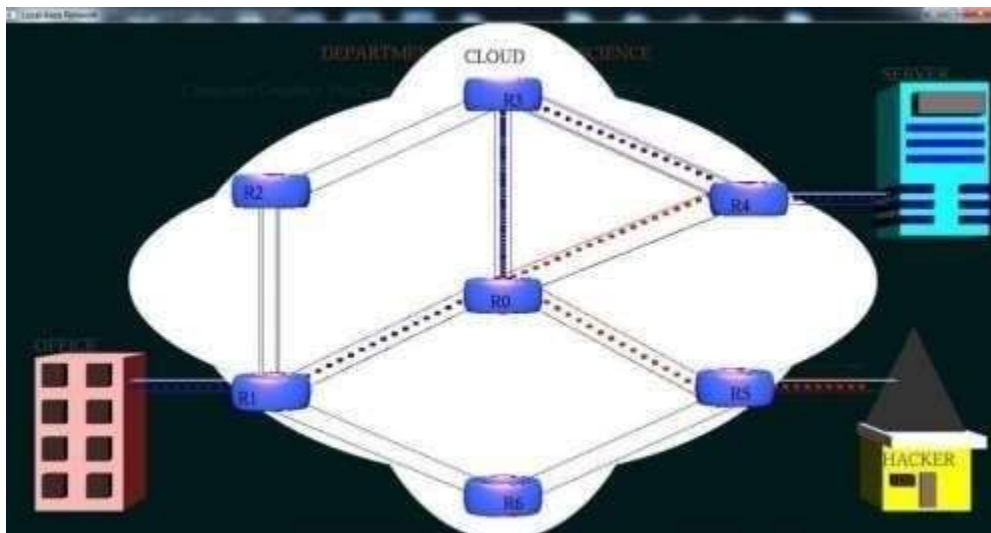


Figure-5.1.4 Explicit Signaling: a node experiences congestion and explicitly sends a packet to the source or destination to inform about congestion.

Conclusion

The simulation and modeling studies reported have enabled us to draw a range of conclusions about the general utility and safety of seven proposed alternate congestion control algorithm for the internet.

We were also able to characterize each of the congestion control algorithm were studied

In the end, we developed some recommendations about to makes sense towards deployment of alternate congestion control.

Algorithms at large scale on the general Internet. Finally, though our study is quite comprehensive, we recognize the need for future work to investigate some questions that we did not tackle.

BIBLIOGRAPHY

WE HAVE OBTAINED INFORMATION FROM MANY RESOURCES TO DESIGN AND IMPLEMENT OUR PROJECT SUCCESSIVELY. WE HAVE ACQUIRED MOST OF THE KNOWLEDGE FROM RELATED WEBSITES. THE FOLLOWING ARE SOME OF THE RESOURCES :

TEXT BOOKS

INTERACTIVE COMPUTER GRAPHICS A TOP-DOWN APPROACH
-By Edward Angel.

GRAPHICS,PRINCIPLES & PRACTICES

- Foley van dam
- Feiner hughes

WEBSITES

<http://jerome.jouvie.free.fr/OpenGL/Lessons/Lesson3.php>
<http://google.com>
<http://opengl.org>